



UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
CÁTEDRA DE PROGRAMACIÓN CONCURRENTE

TRABAJO PRÁCTICO N° 1

“Sistema de reservas de vuelos”

Grupo N° 8

Integrantes		
Nombre	DNI	email
Piñera Nicolas	44195541	nicolas.pinera@mi.unc.edu.ar
Ñáñez Josias Tomas	41886293	josias.nanez@mi.unc.edu.ar
Tabolini Ramiro Alejandro	43884269	ramiro.tabolini@mi.unc.edu.ar
Tarrago Emilio	44198050	emilio.tarrago@mi.unc.edu.ar
Fatu Javier Alejandro	44741112	javier.fatu@mi.unc.edu.ar

Consigna

El presente informe detalla la implementación de una funcionalidad concurrente para gestionar las reservas de asientos en un sistema de reservas de vuelos. Esta funcionalidad se diseñó teniendo en cuenta los siguientes lineamientos y procesos concurrentes:

El sistema debe manejar la reserva de asientos de un avión representado por una matriz donde cada asiento puede estar ocupado, libre o descartado. Se deben mantener cuatro registros distintos para las reservas: pendientes de pago, confirmadas, canceladas y verificadas.

El funcionamiento del sistema posee cuatro etapas:

- **Proceso de Reserva:** Tres hilos se encargan de recibir las solicitudes de reserva de los usuarios. Cada hilo intenta reservar un asiento aleatorio en la matriz, verificando que esté disponible. Si el asiento no está libre, el hilo debe buscar otro asiento que sí lo esté. Una vez reservado el asiento, el mismo se marca como ocupado y se registra la reserva pendiente en el registro de reservas pendientes.
- **Proceso de Pago:** Este proceso es ejecutado por dos hilos, y se encarga de verificar el pago de las reservas pendientes. Cada hilo toma una reserva aleatoria del registro de reservas pendientes y realiza una verificación de pago. Se establece una probabilidad del 90% de que el pago sea aprobado y un 10% de que sea rechazado. Si el pago es aprobado, la reserva se elimina del registro de pendientes, y se agrega al registro de reservas confirmadas; de lo contrario, el asiento pasa a estado descartado mientras que la reserva se marca como cancelada, se elimina del registro de pendientes y se agrega al registro de reservas canceladas.
- **Proceso de Cancelación/Validación:** Si un usuario decide cancelar su reserva, se pasa al proceso de cancelación. Tres hilos se encargan de cancelar las reservas. Cada hilo selecciona una reserva aleatoria de las reservas confirmadas y la cancela con una probabilidad del 10%. Si la reserva es cancelada, se elimina del registro de reservas confirmadas y se agrega al registro de reservas canceladas mientras que el asiento pasa a estado descartado. Si la reserva no es cancelada, la misma se marca como "checked".

- **Proceso de Verificación:** Al finalizar la ejecución, se debe verificar el estado final de la matriz de asientos y los registros de reservas para asegurar que las operaciones se hayan realizado correctamente. Este proceso selecciona de manera aleatoria una reserva del registro de reservas confirmadas. Para cada reserva marcada como “checked”, se debe eliminar del registro de reservas confirmadas y se debe insertar en el registro de reservas verificadas. Este proceso es ejecutado por dos hilos.

Desarrollo

Antes de dar inicio a este trabajo práctico, se aclara que el desarrollo del mismo se realizó en dos fases, la primera se basó en el diseño de diagramas para comprender el funcionamiento principal del sistema y otra que se centró en la implementación práctica de la funcionalidad concurrente para gestionar las reservas de asientos.

Se da comienzo con la elaboración de un diagrama de clases para comprender la estructura y las relaciones principales dentro del sistema de reservas de vuelos. El objetivo de este diagrama fue proporcionar una representación visual de las clases y sus interacciones, lo que permitió una mejor comprensión del diseño general del sistema.

Para este trabajo empezamos realizando una clase "**Asiento**" en donde tenemos diferentes atributos, los cuales son: el estado(libre, ocupado o descartado), un atributo de tipo booleano para verificar si fue chequeado y se inicia en "false", un String relacionado al nombre del hilo que está trabajando en un asiento específico, un usuario que reservara el asiento que se inicia en "null" y una posición que guarda la fila y columna específica donde se ubica el asiento. Al momento de llamar al constructor de la clase Asiento le pasamos por parámetro el número de fila y columna, establecemos el estado en Libre, e inicializamos la posición del asiento creando un objeto de esa clase y pasando los números de fila y columna. Además del constructor tenemos los métodos getter y setter de cada atributo.

La clase **posición** tiene dos atributos enteros que son la fila y columna, que al momento de llamar al constructor se inicializan, luego tenemos los métodos getter de los mismos parámetros.

La clase **usuario** posee dos atributos que son el nombre y el Id de carácter estático, o sea todos los objetos que se creen de la clase usuario comparten la misma Id, por eso cuando se llama al constructor de esta clase se aumenta el valor de la Id y el nombre se conforma de la siguiente manera: "Usuario - " + id , por último tenemos el método getter para el nombre del usuario.

La clase **Avión** fue diseñada con el patrón de diseño singleton que garantiza que tan solo exista un objeto de su tipo y proporciona un único punto de acceso a él para cualquier otro código. Por esto el constructor de esta clase tiene el modificador de acceso "private", en el cual se inicializan los parámetros de cantidad de filas y

cantidad de columnas. Luego los métodos “setInstance” que recibe por parámetro la cantidad de filas y columnas, verifica que no haya otra instancia del avión, la crea y establece el atributo estático “a”. El método “getInstance” comprueba si existe una instancia de avion previa y la devuelve, sino se tira una “IllegalStateException”. Por último tenemos los métodos getter de la cantidad de filas y columnas.

La clase **Reservas de vuelo**, es una clase abstracta la cual guarda todas las variables estáticas que hagan falta y se declaran algunos métodos abstractos, las demás clases (Reservar, Pagar, Verificar, Cancelar) extienden de esta para poder manejar sus variables e implementar los métodos. Posee los siguientes atributos: un avión que se inicia en “null”, una variable booleana que indica si están todos los asientos llenos y se inicializa como “false”, 5 “arraylist”: la primera relacionada a la lista de asientos de nuestro avión, la segunda será la de reservas pendientes de pago la cual guardará los asientos que se reservaron y falten pagar, la tercera la lista de reservas confirmadas la cual guardará los asientos en donde el pago fue aprobado, la cuarta la de reservas canceladas guarda los asientos en donde se anulo la reserva, y la ultima la lista de reservas verificadas donde se guardan todos los asientos que han sido verificados, en todas estas listas se reciben elementos de tipo Asiento. Además de esto, se tienen 3 atributos de tipo semáforo que nos indican el número de hilos que pueden ejecutar cada proceso a la vez (en los 3 solo puede ingresar 1 hilo).

El constructor de la clase Reservas de vuelo solo la primera vez que se instancie llama al método que genera los asientos pasándole por parámetro la instancia de avión creada. En el método “Generar asientos” se inicializa la variable avión, y luego en un bucle for anidado, (donde el primer bucle (variable “i”) llegará hasta el número de filas y el segundo bucle (variable “j”) llegara hasta el número de columnas) agrega a la lista de Asientos un nuevo elemento asiento pasándole como número de fila la variable “i” y número de columna la variable “j”. El unico metodo abstracto que tiene esta clase es “Imprimir Lista”. Tenemos un método que nos entrega un número entero aleatorio de asiento utilizando el método de la clase “Random” “nextInt()” en el cual genera un número aleatorio desde el 0 hasta el número máximo de asiento. Por último tenemos el método que verifica la ocupación, si encuentra algún asiento libre devuelve true, si no devuelve false. Además tenemos los métodos getter de las diferentes listas: Canceladas, Confirmadas, Lista de Asientos, Pendiente de Pago, Verificadas.

La clase **Reservar** que extiende de la clase Reservas de Vuelo modela el proceso de reservar un asiento del avión. Tenemos 2 atributos que son estáticos, es decir que todos los hilos que ejecuten este proceso, utilizaran todos el mismo atributo. El primero es un número entero relacionado al número de clientes esperando por ser atendidos que se inicializa en 0, y el segundo es un semáforo que se le pasa por parámetro el número 1. En los métodos, se comienza sobrescribiendo el método imprimir lista, que utilizando un bucle for each recorriendo la lista de asientos, imprime por pantalla cada asiento con su número de fila y columna.

Luego el método “run” que ejecutarán cada uno de los 3 hilos del proceso de reserva empieza con un bucle while el cual se va a ejecutar siempre que haya algún lugar desocupado. Dentro de este, creamos el asiento que vamos a reservar, luego en un “try / catch” hacemos que el hilo duerma por un tiempo determinado de 240 milisegundos, este tiempo simula el tiempo de duración del proceso de reserva. Luego entramos a un bloque “synchronized” el cual toma el lock de una de las instancias de la clase Llave, el objetivo de este bloque es sincronizar a los hilos para disminuir la cantidad de clientes en 1 si es mayor a 0, si no manda al hilo en ejecución a dormir hasta que lo despierten y salta a la siguiente iteración del while. Luego el hilo que continúa la ejecución entra dentro de otro “try / catch / finally” donde disminuimos en uno al contador del semáforo así ningún otro hilo ingrese en la sección crítica. Se verifica que ya están todos los asientos reservados sale del método run y el hilo queda en estado terminado, si hay asientos libres entra a un bucle “do - while” en donde elige un asiento aleatorio de la lista de asientos, este proceso se va a repetir hasta que encuentre un asiento cuyo estado sea LIBRE, en el finally se libera el recurso tomado por el semáforo. Los finally se colocan al final de los “try - catch” para que se ejecuten si o si al final cuando se produzca una excepción o se salga de la zona en cuestión. Se continúa cambiando el estado del asiento encontrado a OCUPADO, se establece tanto el usuario como el nombre del hilo que reservó dicho asiento. Por último dentro de un “try - catch - finally” se utiliza el semáforo de reserva de pago, se disminuye el contador, y se agrega el asiento reservado a la lista de pendiente de pago, y se libera el semáforo.

La clase **Pagar** que extiende de la clase Reservas de Vuelo modela el proceso de Pago de un asiento del avión. En los métodos, se comienza sobrescribiendo el método imprimir lista, que utilizando un bucle for each

recorriendo la lista de asientos pendientes de pago, imprime por pantalla cada asiento con su número de fila y columna.

En el método “Run” se crean dos objetos, uno de la clase Random y otro de la clase Asiento, luego tenemos un bucle “while” en el cual vamos a permanecer dentro siempre y cuando haya lugares sin reservar o la lista de pendientes de pago no esté vacía, sigue un “try - catch” en donde hacemos dormir al hilo que está ejecutando este proceso 300 milisegundos, tiempo utilizado para simular la duración del proceso. Se ingresa a un bloque synchronized que toma el lock de una de las instancias de llave y sincroniza a todos los hilos que ejecutan el proceso de pago, tenemos un “try - catch - finally” en el cual se disminuye en uno el contador interno del semáforo de pendientes de pago, se verifica si la lista de asientos pendientes de pago no está vacía, si esto se cumple, se le da el valor a nuestro asientos antes definido como un asiento aleatorio de la lista de pendientes de pago, y en la parte “finally” se libera el semáforo. Por último tenemos un bloque “try - catch” en el cual tenemos un bloque synchronized donde sincronizamos tanto a los hilos del proceso de pago como a los de cancelación, tomamos el semáforo de pendientes de pago, eliminamos al asiento seleccionado anteriormente de la lista de pendientes de pago y devolvemos el semáforo, luego tenemos una comprobación en la cual se verifica la probabilidad de que el asiento sea confirmado o cancelado: entonces tomamos el semáforo de confirmadas, agregamos el asiento seleccionado a la lista de confirmadas y devolvemos el semafor, si no se cumple, tomamos el semáforo de canceladas, agregamos el asiento a la lista de canceladas, devolvemos el semáforo, establecemos el estado del asiento en DESCARTADO y al usuario en NULL.

La clase **Cancelar** que extiende de la clase Reservas de Vuelo modela el proceso de Cancelación de un asiento del avión. En los métodos, se comienza sobreescribiendo el método imprimir lista, que utilizando un bucle for each recorriendo la lista de asientos cancelados, imprime por pantalla cada asiento con su número de fila y columna. Tenemos un método que verifica cada uno de los asientos en la lista de confirmados y devuelve “true” si algún asiento no fue chequeado.

El método “Run” crea una variable auxiliar de tipo Random, luego entramos en un bucle “while” el cual se va a repetir siempre y cuando haya algún asiento no chequeado, que la lista de pendientes de pago no esté vacía o si hay algún asiento libre. Entrando en el bucle, nos encontramos un bloque “try - catch” en el cual

mandamos el hilo que realiza este proceso 120 milisegundos para simular el proceso de cancelación. Posteriormente inicializamos un asiento el cual vamos a cancelar, tenemos un bloque synchronized con el cual sincronizamos todos los hilos que ejecutan el proceso de cancelación, dentro del mismo tenemos un bloque “try - catch - finally” en el cual, se toma el semáforo de confirmadas, se verifica si hay algún asiento sin verificar, y dentro de un bucle “do - while” se selecciona un asiento aleatorio de la lista de confirmados, dicho bucle se va a repetir hasta que el asiento seleccionado no haya sido chequeado, y se libera el semáforo de confirmadas. Por último tenemos una verificación donde si el asiento está dentro del 90% se establece que fue chequeado, si no se cumple, entramos en un bloque “try - catch” dentro del mismo encontramos un bloque synchronized para sincronizar los hilos de cancelar con los de pagar, tomamos el semáforo de confirmadas, removemos el asiento de la lista de confirmadas, devolvemos el semáforo, tomamos el semáforo de canceladas, agregamos el asiento a la lista de cancelados y devolvemos el semáforo, establecemos el nombre de usuario en NULL y el estado del asiento en DESCARTADO.

La clase **Verificar** que extiende de la clase Reservas de Vuelo modela el proceso de Verificación de un asiento del avión. En los métodos, se comienza sobreescribiendo el método imprimir lista, que utilizando un bucle for each recorriendo la lista de asientos verificados, imprime por pantalla cada asiento con su número de fila y columna. Tenemos un método que verifica cada uno de los asientos en la lista de confirmados y devuelve “true” si algún asiento no fue chequeado. El método validación recibe por parámetro un asiento, y en cada verificación se comprueba el estado del asiento, si se encuentra que el asiento está LIBRE larga una excepción de estado ilegal y muestra por pantalla “Hay asientos libres”, si el asiento no tiene asignado un usuario y el estado es distinto de descartado, larga una excepción de estado ilegal y muestra por pantalla “Asiento Verificado sin usuario”, si el asiento pertenece a la lista pendiente de pago, larga una excepción de estado ilegal y muestra por pantalla “Asiento Contenido en pendiente de pago”, si el asiento está en la lista de asientos confirmados, larga una excepción de estado ilegal y muestra por pantalla “Asiento contenido en confirmadas”, si el asiento no está chequeado y su estado no es DESCARTADO larga una excepción de estado ilegal y muestra por pantalla “Asiento no Chequeado”.

En el método “Run” tenemos un bucle “while” en el cual vamos a permanecer siempre y cuando siempre haya lugares sin ocupar o la lista de asientos pendientes de pago no está vacía o la lista de confirmados no está vacía, dentro tenemos un asiento el cual vamos a verificar, hay un bloque “try - catch” donde mandamos a dormir al hilo 300 milisegundos para simular el proceso de verificar, tenemos un bloque synchronized para sincronizar los hilos que realizan el proceso de verificar, dentro del mismo tenemos un bloque “try - catch” donde tomamos el semáforo de confirmadas, hay una verificación que comprueba si hay algún asiento en la lista de confirmadas sin chequear, si se cumple se toma un asiento aleatorio de la lista de confirmadas, esto se repite hasta obtener un asiento que no haya sido chequeado, y se devuelve el semáforo de confirmada. Luego tenemos otro bloque “try - catch” en el cual el asiento seleccionado se agrega a la lista de verificación, se toma el semáforo de confirmadas, se saca el asiento de la clase confirmadas y se devuelve el semáforo, por último se llama al método validar pasándole por parámetro el asiento seleccionado para corroborar que no genera ningún problema, si esto pasa lo informa en el log.

Por otro lado el programa cuenta con la implementación de un **Log** a fines estadísticos, el cual escribe en un archivo de texto cada 200 milisegundos, el estado de los asientos en ese momento, usamos para su implementación la clase Logger, al cual le pusimos un handler para indicar la ruta de salida de los datos, el cual es escribir en dicho archivo “Log.txt”, y cuenta con un método estático que devuelve el Logger, para que cualquier clase pueda mandar información (en este caso lo usamos con verificar, para informar de algún problema que pueda existir luego de la ejecución) y por último al final de la ejecución el Logger escribe en el archivo la duracion total de la ejecucion.

La clase **Llave** también se diseñó con el patrón singleton, y se utiliza para crear objetos los cuales serán utilizados en diversos bloques “synchronized”. Tenemos 5 llaves creadas y luego el método “getInstance” de cada una de las llaves.

La clase **Generador de usuarios** implementa la interfaz “Runnable”, posee un ArrayList de Usuarios de carácter estático y tiene un método getter de la misma lista. En el método “run” encontramos un bucle “while” que se va a repetir siempre que no estén todos los asientos ocupados, dentro del mismo tenemos un “try - catch” en donde hacemos dormir al hilo 140milisegundos simulando el proceso de

creación de un usuario, luego tenemos un bloque synchronized para ingresar a la sección crítica para que no ingrese más de un hilo y modifique al mismo tiempo la lista de usuarios. En el mismo se verifica si los clientes en espera son menores a la cantidad de asientos, si esto es verdad, se genera y agrega un nuevo usuario a la lista y se aumenta en 1 los clientes en espera para reservar. Luego se verifica si los clientes en espera para reservar son igual o mayor que la cantidad de asientos, si se cumple la variable booleana "lleno" se establece en True y no genera más usuarios. Por último se llama a través de la llave se notifica a algún hilo de reservar una vez generado el usuario para que se despierte y ejecute la reserva.

La última clase que se va a explicar en este trabajo será la **Principal o Main**. Tenemos el método principal donde se crea nuestro avión, se llama a la función "generar avión", se crea un hilo que ejecuta el proceso de generación de usuarios, otro que ejecutara el proceso del "Log" y un "ArrayList" de hilos que ejecutan cada uno de los procesos, se llaman a las funciones "generar hilos para Verificar" , "generar hilos para Reservar", "generar hilos para Pagar", "generar hilos para Cancelar" , "Inicializar hilos", en todas se pasa por parámetro nuestro "ArrayList" de hilos, se lanzan tanto el hilo del generador de usuarios como el del "log", sigue un "try - catch" donde el hilo del "log" utiliza el método "join" para que el hilo principal espere a que el log termine de ejecutarse, y se lo manda a dormir 500 milisegundos, para dar tiempo a las comprobaciones finales de verificar y para finalizar se manda al "log" el estado de cada uno de los hilos.

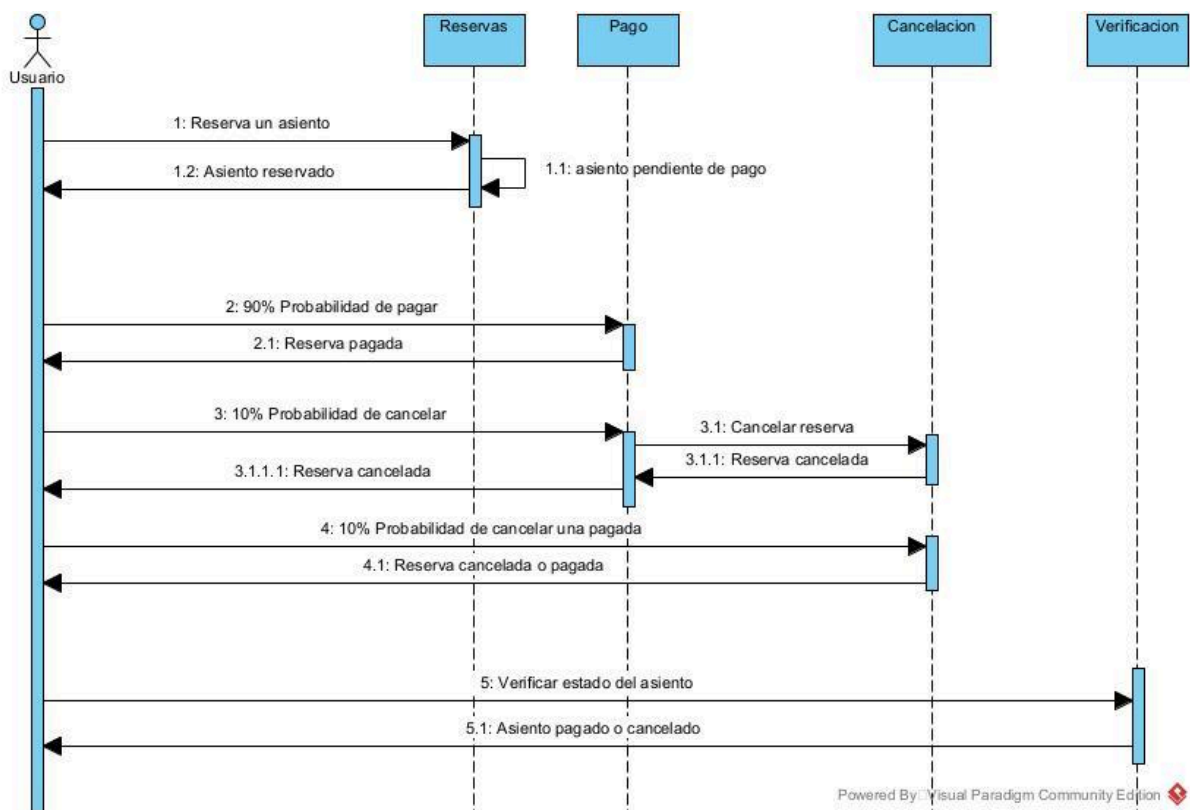
Luego del método principal sigue la función generar hilos para Verificar la cual recibe por parámetro un ArrayList de hilos, la misma agrega dos hilos, los cuales ejecutarán el proceso de verificación, la función Generar Hilos para Reserva agrega 3 hilos que ejecutan el proceso de reserva, Generar Hilos para Pagar agrega 2 hilos que ejecutan el proceso de pago, Generar Hilos para Cancelar agrega 3 hilos que ejecutan el proceso de cancelación, Inicializar hilos lanza todos los hilos dentro del arraylist.

El método Generar Avión lo que hace es preguntarle al usuario si desea usar el número de filas y columnas por defecto para su avion, si es así la instancia del avión se inicializa con 6 filas y 31 columnas, si no le pide que ingrese un numero entero mayor a cero para ambas cantidades y se las pasa a la instancia. El método Mostrar estado hilos, crea un string y una variable auxiliar entera, en un bucle for each, recorre todo el arraylist de hilos, y arma una sentencia la cual muestra el

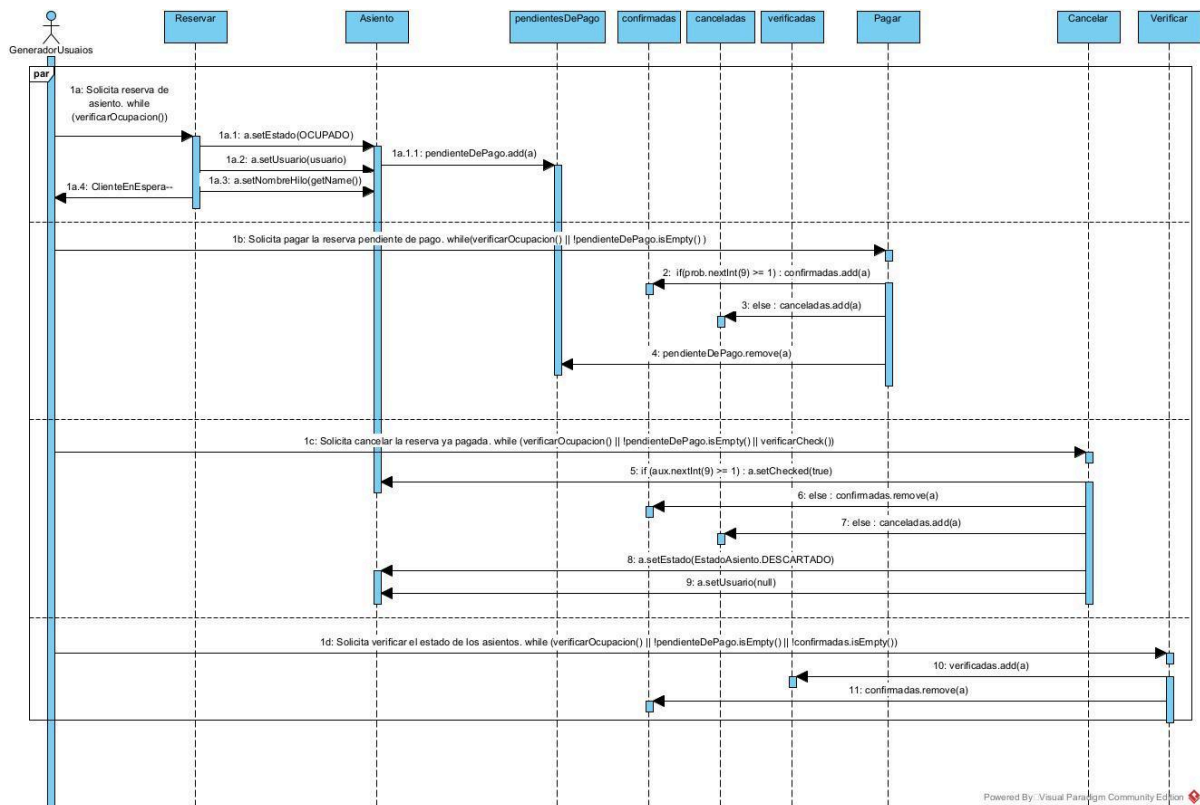
número de hilo y el estado en el que termino, esta misma sentencia es pasada al hilo del Log para que también lo muestre por pantalla.

Se da comienzo con la elaboración de un diagrama de secuencias para comprender las interacciones temporales entre los objetos y las clases del sistema de reservas de vuelos.

Se identifican los *actores principales*, como usuarios del sistema y el propio sistema de reservas de vuelos. También se identifican *interacciones temporales* durante el proceso, incluyendo la solicitud de reserva, el pago, la cancelación y la verificación.



El usuario comienza reservando un asiento, luego hay una alta probabilidad(90%) de que el usuario proceda con el pago de la reserva. También existe una probabilidad(10%) de que el usuario cancele la reserva. En el caso de que pague la reserva, existe una probabilidad(10%) de que la cancele a la reserva luego de realizar el pago. Por último se verifica el estado de la reserva, si está pagada o si está cancelada.



Conclusiones:

El diseño e implementación del sistema de reservas de vuelos han demostrado ser un proyecto integral y educativo en el campo de la programación concurrente. Este sistema ha ampliado nuestro entendimiento de conceptos fundamentales, tales como el manejo de concurrencia, la sincronización de procesos y la gestión eficiente de recursos compartidos.

Además, el análisis detallado de los requisitos del sistema y la planificación cuidadosa de su estructura utilizando los diagramas planteados han sido cruciales para realizar el proyecto.

Durante el proceso de implementación, se enfrentaron diversos desafíos que requirieron trabajar en equipo para comprender el problema y llegar a una solución sin sacrificar de manera significativa el rendimiento del programa. También cabe destacar lo importante de la gestión adecuada de la concurrencia y la prevención de condiciones de carrera para garantizar la integridad de los datos del sistema.

Como análisis de los tiempos podemos estimar que nuestro programa debería ejecutarse en un tiempo alrededor de los 27.9 segundos este tiempo lo

obtenemos calculando el tiempo que tarda en pagar ya que este es nuestro eslabón más lento entonces se genera un cuello de botella, por ende usamos ese proceso para calcular cuanto tardaria el programa, esto como podemos apreciar a la hora de ejecutar nuestro código este tiene un tiempo de ejecución alrededor de 28,808 segundos, lo cual tiene bastante relación. Lo cual es bastante positivo ya que no está discrepando tanto del tiempo teórico como del tiempo real.

N	T. de ejecución [s]
1	28,899
2	28,773
3	29,242
4	28,526
5	28,499
6	28,514
7	28,522
8	28,515
9	29,77
10	28,818
T. Promedio	28,8078